The Journal of Open Source Software

GAIO.jl - a concise Julia package for the global analysis of dynamical systems

April Herwig [1], **Oliver Junge** [1], **and Michael Dellnitz** [2]

1 Technical University of Munich, Germany **2** University of Paderborn, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

We provide an implementation of set-oriented numerical methods (Dellnitz et al., 2001; Dellnitz & Junge, 2002) in a Julia package. In the context of dynamical systems, the package enables the rigorous computation of invariant sets (e.g. chain recurrent sets, attractors and invariant manifolds) and provides discretizations of the transfer and the Koopman operator, enabling the computation of, e.g., invariant measures, almost invariant, cyclic and coherent sets. The redesign of the original implementation (Junge et al., 2015) in Julia presented in this note has a more concise syntax, while showing the same or even better performance.

Statement of Need

A *dynamical system* is a mathematical model of a system whose state evolves with time. Key questions about the possible evolutions in a system include whether certain subsets of state space are invariant, the stability of these subsets as well as questions about statistical properties of typical trajectories. Such topological and statistical questions (among others) can be answered using *set-oriented* numerical techniques (Dellnitz et al., 2001; Dellnitz & Junge, 2002), see also (Hsu, 1987; Mischaikow, 2002).

This note presents an implementation of a set-oriented approach to numerical computations (Dellnitz et al., 2001; Dellnitz & Junge, 2002) in the Julia language, encapsulated in the package GAIO.jl. The data structures and the algorithmic interface have been completely redesigned over the original implementation in C/MATLAB (Junge et al., 2015). As a result, the code is much more concise and closer to the mathematical formulation of the underlying algorithms. At the same time, the performance is equal or better. We showcase the features of GAIO.jl by some classical computations of (almost) invariant sets and invariant measures in dynamical systems.

Global Analysis of Dynamical Systems

A dynamical system in which time is modelled in discrete steps is given by a map $f : X \to X$ on some domain X. For simplicity, we here assume X to be a compact subset of $\mathbb{R}^d$ and f to be a homeomorphism, i.e. bijective and continuous with a continuous inverse (but most concepts also work under weaker assumptions). We recall some basic notions from dynamical systems required for the subsequent examples.

The Journal of Open Source Software

Geometric/topological analysis

A set $S \subset X$ is *forward invariant* if $f(S) \subset S$, *backward invariant* if $f^{-1}(S) \subset S$ [1], and *invariant* if it is both forward and backward invariant. An invariant set S is called *attracting* if there exists a neighborhood $U \supset S$ such that for any other neighborhood $V \supset S$ there exists a $k_0 \in \mathbb{N}$ such that $f^k(U) \subset V$ for all $k \geq k_0$. The *maximal invariant set* contained in some set S is

$$\mathrm{Inv}(S) = \{x \in S \mid f^k(x) \in S \text{ for all } k \in \mathbb{Z}\}.$$

It follows immediately from the definition that $\mathrm{Inv}(S)$ contains all other invariant sets which are contained in S. The following proposition is important for its computation.

Proposition 1: (Easton, 1998) If $S \subset X$ is forward invariant, then $\mathrm{Inv}(S) = \bigcap_{k\geq 0} f^k(S)$.

If S is not forward invariant, the set $A_S := \bigcap_{k\geq 0} f^k(S)$ is called the *attractor relative to* S (Dellnitz & Hohmann, 1997). This characterization leads to a natural algorithm for approximating A_S by repeatedly tightening a cover by finite collections of subsets of X (Dellnitz & Hohmann, 1997):

Algorithm 2: Given a partition $\mathcal{X}$ of X into (essentially) disjoint sets and some covering $\mathcal{A} \subset \mathcal{X}$ of A_S, repeat the following two steps until a prescribed diameter of the partition elements is reached:

1. Refine $\mathcal{X}$ into a strictly finer partition $\mathcal{X}'$ (i.e. such that $\mathrm{diam}(\mathcal{X}') \leq \theta \cdot \mathrm{diam}(\mathcal{X})$ for some fixed $\theta < 1$). Let $\mathcal{A}'$ be the corresponding refinement of the covering $\mathcal{A}$ of A_S.
2. Map the refined covering forward under f, i.e. cover [2] $f(|\mathcal{A}'|)$ by elements of $\mathcal{X}'$. Intersect this covering with $\mathcal{A}'$.

In GAIO.jl, partitions resp. coverings are implemented as collections of *boxes*, i.e. multidimensional intervals of the form $[\ell_1, u_1] \times \ldots \times [\ell_d, u_d]$.

Example: A four wing attractor

We now implement the above algorithm in GAIO.jl and then use it to compute a covering of the attractor of the following system (Wang et al., 2009):

$$\begin{aligned}\dot{x} &= ax + yz\\ \dot{y} &= dy + bx - zy\\ \dot{z} &= -z - xy\end{aligned}$$

where $a, b, d \in \mathbb{R}$ are parameters. Let f be the time-2 flow map of the system, discretized using 20 steps of the standard Runge-Kutta fourth order method:

```
using GAIO
const a, b, d = 0.2, -0.01, -0.4
v((x,y,z)) = @. (a*x+y*z, d*y+b*x-z*y, -z-x*y)
f(x) = rk4_flow_map(v, x, 0.01, 20)
```

We begin by computing a long trajectory, starting at some point close to the attractor:

```
x0 = (0.94, -2.2, 0.4)
trajectory = [x0]
for _ in 1:100_000
    x = trajectory[end]
    push!(trajectory, f(x))
end
lines(trajectory)
```

[1] $f^{-1}(S) = \{x \in X : f(x) \in S\}$ denotes the *preimage* of S.
[2] $|\mathcal{A}| = \bigcup_{\chi\in\mathcal{A}} \chi$

DRAFT

Figure 1 shows the resulting trajectory. Of course, any other package for solving initial value problems can be used in order to approximate the flow map. Note, however, that later we will need to evaluate f on many initial conditions, so efficiency of the solver matters.

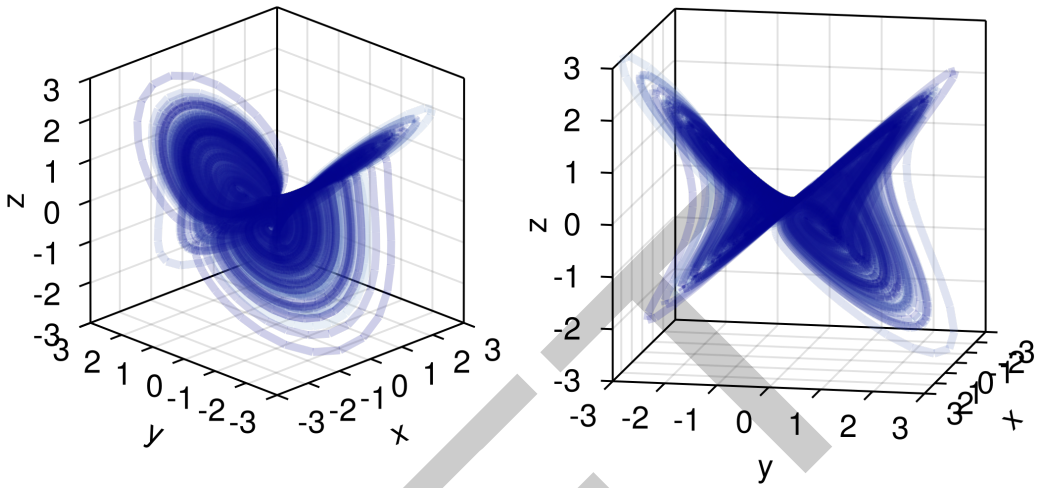

Figure 1: A trajectory of the four wing system.

For step 1. in Algorithm 2, we construct a `BoxGrid` `PX` which partitions the domain box $X = [-5, 5]^3$ into a $2 \times 2 \times 2$ grid of boxes. One can think of `PX` as a representation of the power set[3] $\mathcal{P}(\mathcal{X})$ of the partition $\mathcal{X}$:

```
center, radius = (0.,0.,0.), (5.,5.,5.)
X = Box(center, radius)
PX = BoxGrid(X, (2, 2, 2))
```

We next construct a (very coarse) initial covering, namely the `BoxSet` `A` consisting of all boxes in `PX`, covering the entire domain X:

```
A = cover(PX, :)
```

The command

```
subdivide(A, k)
```

could then be used for refining the `BoxSet` `A` by bisecting each box in the k-th coordinate direction.

In order to implement step 2. in Algorithm 2, we first note that the flow map f induces a map $F : \mathcal{P}(\mathcal{X}) \to \mathcal{P}(\mathcal{X})$ via

$$F(\{\chi\}) = \{\hat{\chi} \in \mathcal{X} \mid \hat{\chi} \cap f(\chi) \neq \emptyset\}\,.$$

[3]The *power set* $\mathcal{P}(S)$ of some set S is the set of all subsets of S.

DRAFT

The Journal of Open Source Software

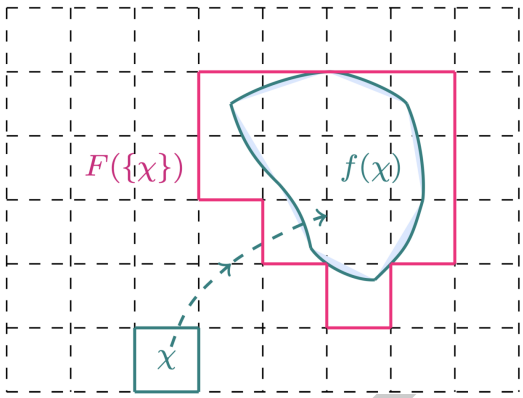

Figure 2: The map $F : \mathcal{P}(\mathcal{X}) \to \mathcal{P}(\mathcal{X})$ induced by $f : X \to X$.

GAIO.jl provides multiple methods for approximating the map F. An intuitive method is to randomly sample some points (from a uniform distribution) within a box, and then map each point with f. Such an approximation to the map F can be instantiated by

```
F = BoxMap(:montecarlo, f, PX)
```

If a rigorous (outer) cover of the image $F(\{\chi\})$ is required, special sampling techniques (Junge, 2000) and/or interval arithmetic can be used. We can now implement the above algorithm:

```
function relative_attractor(F, A; steps)
    for k in 1:steps
        A = subdivide(A, k%3+1)
        A = F(A) ∩ A
    end
    return A
end
```

In each iteration we cycle through the coordinate direction in which we bisect the boxes of the current covering. Compare with the implementation in Matlab below. We invoke this function on the initial covering A, resulting in the covering of the attractor shown in Fig. 3:

```
A = relative_attractor(F, A, steps=21)
plot(A)
```

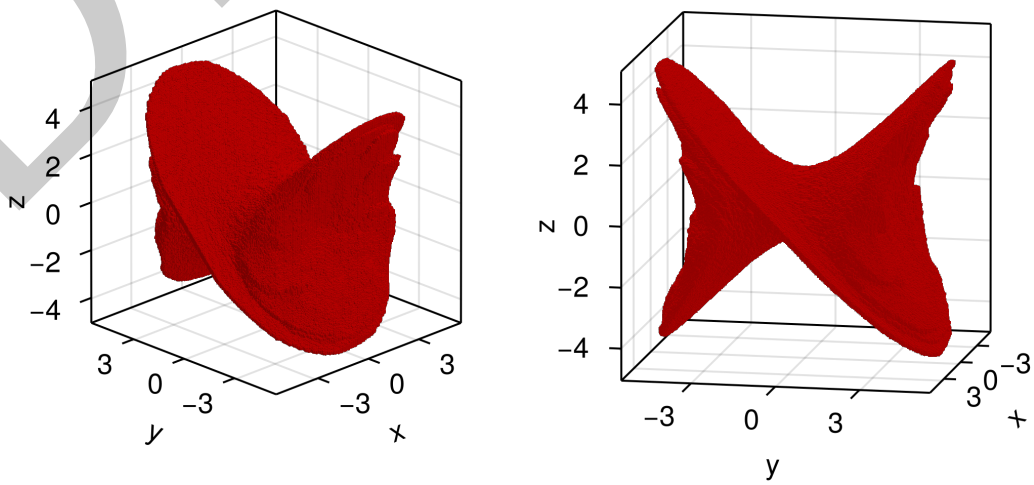

Figure 3: Attractor of the four wing system.

DRAFT

Other algorithms implemented in GAIO.jl include ones for, e.g., computing forward, backward and maximal invariant sets, (un-)stable manifolds, chain-recurrent sets and Morse decompositions.

Statistical analysis

The map $f : X \to X$ induces a map $f_\sharp : \mathcal{M} \to \mathcal{M}$ on measures[4] via

$$f_\sharp \mu := \mu \circ f^{-1}.$$

This is a bounded linear Markov operator, the *Perron-Frobenius* or *transfer operator*. Much information about macroscopic features of the dynamics of f can be extracted from eigenmeasures of $f_\sharp$ at eigenvalues with modulus close to one (Dellnitz & Junge, 1998).

In particular, in our setting there exists (Kryloff & Bogoliouboff, 1937) an eigenmeasure $\mu = f_\sharp \mu$ at the eigenvalue 1 of $f_\sharp$, an *invariant measure*. A *natural* invariant measure (Young, 2002) quantifies the statistics of typical trajectories: regions of phase space which are visited more often by such trajectories receive more μ-mass.

Discretization

One can approximate some $\mu \in \mathcal{M}$ by a discrete measure

$$\mu_g(S) = \sum_{j=1}^{n} g_j \frac{m(\chi_j \cap S)}{m(\chi_j)},$$

where $\{\chi_1, \chi_2, \dots, \chi_n\}$ is a subset of the partition $\mathcal{X}$ (e.g. the covering that we computed above), $g = (g_1, \dots, g_n) \in \mathbb{C}^n$ and m is Lebesgue measure on $\mathbb{R}^d$. The coefficients g of an approximate invariant measure μ_g should then satisfy

$$g_i = \mu_g(\chi_i) \overset{!}{=} f_\sharp \mu_g(\chi_i) = \sum_{j=1}^{n} g_j \underbrace{\frac{m(\chi_j \cap f^{-1}(\chi_i))}{m(\chi_j)}}_{=:(F_\sharp)_{ij}}.$$

The matrix $F_\sharp \in \mathbb{R}^{n \times n}$ defines a Markov chain on the mentioned subset of $\mathcal{X}$ and is our finite approximation of $f_\sharp$ on $\mathcal{M}_n = \{\mu_g : g \in \mathbb{C}^n\}$. Convergence of the spectrum of this approximation (known as *Ulam's method*) can be shown by considering a small random perturbation of the original map f (Dellnitz & Junge, 1998). The matrix $F_\sharp$ can be computed similarly to how the image for F is computed, e.g. by approximating the transition probabilities $(F_\sharp)_{ij}$ using random sample points.

As an example, we compute $F_\sharp$ on the covering of the attractor constructed above and then use the `eigs` function from Arpack.jl in order to compute part of the spectrum of $F_\sharp$, which is shown in Fig. 4.

```
T = TransferOperator(F, A, A)
lambda, ev, n_converged = eigs(T)
scatter(lambda)
```

[4] $\mathcal{M}$ denotes the space of finite, complex valued Borel measures on X.

DRAFT

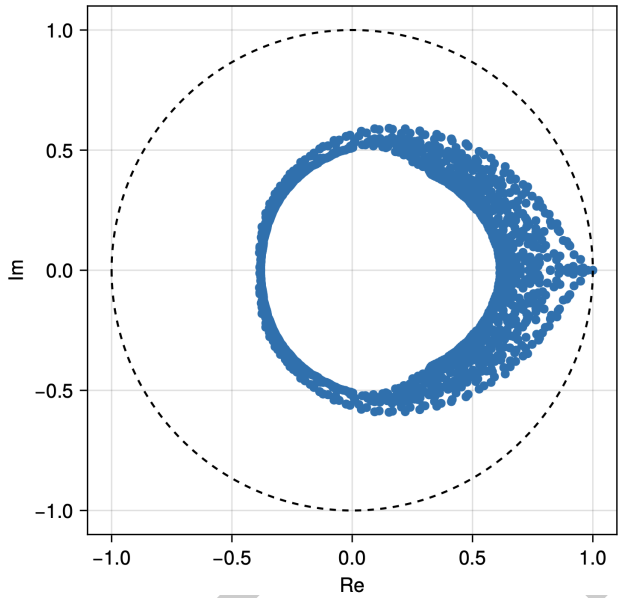

Figure 4: Spectrum of $F_\sharp$.

The eigenvalue 1 is simple, the corresponding approximate invariant measure is shown in Fig. 5.

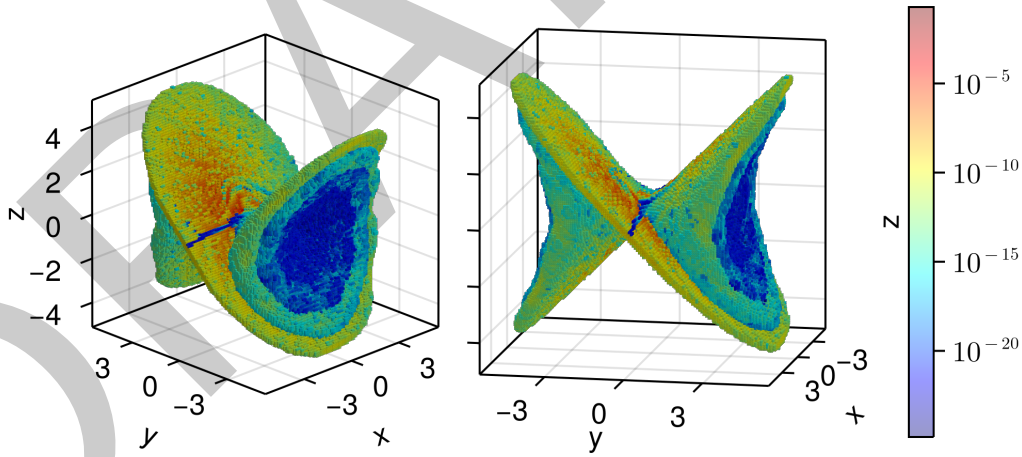

Figure 5: Natural invariant measure of f. Compare with Fig. 1: the "centers" of the wings are visited frequently, correspondingly they have more μ-mass. The "outer spirals" are visited infrequently, they have less μ-mass.

In Fig. 6, we further show the eigenmeasure at the second largest real eigenvalue $\lambda \approx 0.978$. By its sign structure, this measure decomposes the attractor into two almost invariant sets (Dellnitz & Junge, 1998), i.e. two sets A_-, A_+ for which the invariance ratio $m(A_+ \cap f^{-1}(A_+))/m(A_+)$ (resp. A_-) is close to 1.

DRAFT

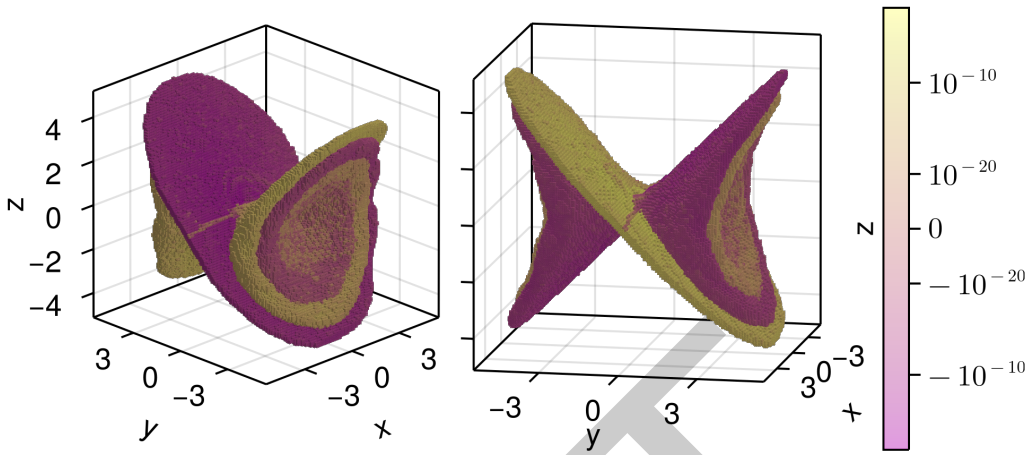

Figure 6: Eigenmeasure corresponding to the second-largest eigenvalue 0.978. The sign structure of the measure decomposes the attractor into two almost invariant sets.

GAIO in the Julia Language

Philosophy

The data structures and algorithms that make up GAIO were originally developed in the 90's and written in C (Dellnitz et al., 2001). An interface was written in Python which made use of the Numerical Python environment (Harris et al., 2020), and 3D plotting was done by writing files which were then read by the dedicated visualization software GRAPE (Rumpf & Wierse, 1992). Later, a second interface was written in MATLAB. It comes as no surprise that this architecture was hard to maintain (see Fig. 7). For this reason, the architecture was stripped to just the core (in C) and the Matlab interface in 2015. Still, GAIO was a picture book incarnation of the *two language problem* (Bezanson et al., 2017).

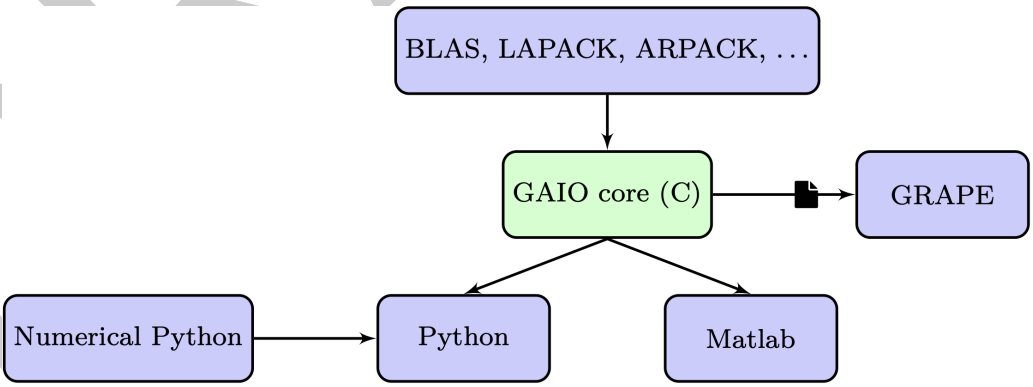

Figure 7: An earlier software architecture.

GAIO was fully redesigned and rebuilt in the Julia language starting in 2020. The reason to do so (and the decision to use Julia) was twofold:

- *Solving the two language problem.* The set-oriented techniques require too many evaluations of the map f to be reasonably implemented in an interpreted language. As with many scientific computing applications, compilation into fast machine code is necessary if one wishes to apply the methods to a non-trivial problem. However, the code should still be easy to write and read.

- *Abstraction of the syntax.* The original GAIO code required the user to be aware of the details of the internal data structures. For example, box collections were stored in a binary tree and flags in the nodes of that tree were used in order to implement the box map $F : \mathcal{P}(\mathcal{X}) \to \mathcal{P}(\mathcal{X})$. Here is the original GAIO/MATLAB implementation of the Algorithm 2 for computing the relative global attractor:

```
function relative_attractor(tree, f, steps)
    for i = 1:steps,
        tree.set_flags('all', to_be_subdivided);
        tree.subdivide(to_be_subdivided);
        b = tree.boxes(-1);
        while (~isempty(b))
            c = b(1:d);
            r = b(d+1:2*d);
            P = X*diag(r)+ones(size(X))*diag(c);
            tree.set_flags(f(P)', hit);
            b = tree.next_box(-1);
        end
        tree.remove(hit);
    end
```

In contrast, in GAIO.jl, details on how box collections are stored and how the box map $F : \mathcal{P}(\mathcal{X}) \to \mathcal{P}(\mathcal{X})$ is realized are essentially completely hidden from the user. After defining F as a `BoxMap`, it can simply be applied to a `BoxSet`, resulting in another `BoxSet`. Since `BoxSet` is a subtype of `AbstractSet`, all set operations can be applied. As a result, the implementation of set-oriented algorithms - like the algorithm for computing relative attractors - is very close to its mathematical formulation.

Fitting into Julia's ecosystem

Another reason for the decision to use Julia for GAIO.jl is the large (and growing) open source scientific computing community. GAIO.jl makes extensive use of linear algebra routines (LinearAlgebra.jl (Bezanson et al., 2017)), sparse arrays (SparseArrays.jl (Bezanson et al., 2017), Arpack.jl (Lehoucq et al., 1998)), graph/network routines (MatrixNetworks.jl (Nassar et al., 2021)), and numerical integrators (DifferentialEquations.jl (Rackauckas & Nie, 2017)), plotting ecosystems (Makie.jl (Danisch & Krumbiegel, 2021), Plots.jl (Christ et al., 2023)), among others.

A particular example of the effectiveness of such an open source scientific computing model is CUDA.jl. The sample-point methods for cell mapping are examples of so-called *embarrassingly parallel* (Herlihy et al., 2020) problems. It has therefore been a long-standing desire to utilize the GPU to perform such massively parallel algorithms in GAIO. However, under the previous architecture this would have to be written in CUDA's native C interface, which suffers the issues mentioned in the preceeding section.

This is solved by the amazing work done to create CUDA.jl. The cell mapping can be written as a generic kernel whose length (in lines of code) is the same as the standard code, meaning the algorithms that make up GAIO.jl can receive up to a 200-fold (Herwig, 2022) performance boost without ever sacrificing readability.

Furthermore, the open-source nature of Julia's ecosystem has helped in programming GAIO.jl countless times. Since code is freely available, code reuse is common among the Julia community.

DRAFT

Conclusion

The Julia package GAIO.jl is introduced via an example of a three dimensional dynamical system. The software has been redesigned and rebuilt from the ground up to balance high performance and elegance, no longer needing to rely on two languages to do so. Future work is planned to use these structures e.g. for homology computation in cubical complexes, as well as to even more tightly integrate into the existing scientific computing ecosystem.

Bezanson, J., Edelman, A., Karpinski, St., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Christ, S., Schwabeneder, D., Rackauckas, C., Borregaard, M. K., & Breloff, T. (2023). Plots.jl – a user extendable plotting API for the Julia programming language. *Journal of Open Research Software*. https://doi.org/https://doi.org/10.5334/jors.431

Danisch, S., & Krumbiegel, J. (2021). Makie.jl: Flexible high-performance data visualization for Julia. *Journal of Open Source Software*, *6*(65), 3349. https://doi.org/10.21105/joss.03349

Dellnitz, M., Froyland, G., & Junge, O. (2001). The algorithms behind GAIO – set oriented numerical methods for dynamical systems. In *Ergodic theory, analysis, and efficient simulation of dynamical systems* (pp. 145–174, 805–807). Berlin: Springer. https://doi.org/10.1007/978-3-642-56589-2_7

Dellnitz, M., & Hohmann, A. (1997). A subdivision algorithm for the computation of unstable manifolds and global attractors. *Numer. Math.*, *75*(3), 293–317. https://doi.org/10.1007/s002110050240

Dellnitz, M., & Junge, O. (1998). On the approximation of complicated dynamical behavior. *SIAM J. Numer. Anal.*, *36*(2), 491–515. https://doi.org/10.1137/S0036142996313002

Dellnitz, M., & Junge, O. (2002). Set oriented numerical methods for dynamical systems. In *Handbook of dynamical systems. Volume 2* (pp. 221–264). Amsterdam: Elsevier. https://doi.org/10.1016/s1874-575x(02)80026-1

Easton, R. W. (1998). Dynamical Systems. In *Geometric Methods for Discrete Dynamical Systems*. Oxford University Press. https://doi.org/10.1093/oso/9780195085457.003.0002

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Herlihy, M., Shavit, N., Luchangco, V., & Spear, M. (2020). *The art of multiprocessor programming*. Elsevier. https://doi.org/10.1016/C2011-0-06993-4

Herwig, A. (2022). *GAIO.jl: Set-oriented methods for approximating invariant objects, and their implementation in Julia. B.sc. Thesis, Technical University of Munich*. https://github.com/April-Hannah-Lena/schoolwork/blob/2eada059678d91bad8a813c3e6b657a1ac72e86f/Thesis/main.pdf

Hsu, C.-S. (1987). *Cell-to-cell mapping. A method of global analysis for nonlinear systems* (Vol. 64). Springer, Cham. https://doi.org/10.1007/978-1-4757-3892-6

Junge, O. (2000). Rigorous discretization of subdivision techniques. In *Proceedings of Equadiff '99, Berlin, Germany, Vol. 2* (pp. 916–918). Singapore: World Scientific. https://doi.org/10.1142/4469

Junge, O., Herwig, A., & others. (2015). GAIO (Global Analysis of Invariant Objects). In *GitHub repository*. https://github.com/gaioguy/GAIO; GitHub.

Kryloff, N., & Bogoliouboff, N. (1937). La théorie générale de la mesure dans son application

DRAFT

à l'étude des systèmes dynamiques de la mécanique non linéaire. *Ann. Math. (2)*, *38*, 65–113. https://doi.org/10.2307/1968511

Lehoucq, R. B., Sorensen, D. C., & Yang, C. (1998). *ARPACK users' guide: Solution of large-scale eigenvalue problems with implicitly restarted arnoldi methods*. SIAM. https://doi.org/10.1137/1.9780898719628

Mischaikow, K. (2002). Topological techniques for efficient rigorous computation in dynamics. *Acta Numerica*, *11*, 435–477. https://doi.org/10.1017/S0962492902000065

Nassar, H., Gleich, D., Hyatt, K., Meher, P. C., Benson, A., Kelman, T., Kröger, O., Eikmeier, N., Saba, E., & Kim, J. (2021). *MatrixNetworks.jl v1.0.2*. Zenodo. https://doi.org/10.5281/zenodo.4560914

Rackauckas, C., & Nie, Q. (2017). DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *The Journal of Open Research Software*, *5*(1). https://doi.org/10.5334/jors.151

Rumpf, M., & Wierse, A. (1992). GRAPE, eine objektorientierte Visualisierungs– und Numerikplattform. *Informatik, Forschung Und Entwicklung*, *7*, 145–151.

Wang, Z., Sun, Y., Wyk, B. J. van, Qi, G., & Wyk, M. A. van. (2009). A 3-d four-wing attractor and its analysis. *Brazilian Journal of Physics*, *39*(3), 547–553. https://doi.org/10.1590/S0103-97332009000500007

Young, L.-S. (2002). What are SRB measures, and which dynamical systems have them? *J. Stat. Phys.*, *108*(5-6), 733–754. https://doi.org/10.1023/A:1019762724717

DRAFT